

Dataproc Serverless

來源：<https://codelabs.developers.google.com/dataproc-serverless?hl=zh-tw>

步驟數：8

在這個程式碼研究室中，您將全面瞭解 Dataproc Serverless 的所有資訊，包括如何開始使用及存取豐富的功能集。

目錄

1. [總覽 - Google Dataproc](#)
2. [設定](#)
3. [使用 Dataproc Batches 執行無伺服器 Spark 工作](#)
4. [Dataproc 指標和觀測能力](#)
5. [Dataproc 範本：BigQuery -> Cloud Storage](#)
6. [Dataproc 範本：CSV 轉 Parquet](#)
7. [清除所用資源](#)
8. [後續步驟](#)

1. 總覽 - Google Dataproc

[Dataproc](#) 是一項全代管且具備高擴充性的服務，可執行 Apache Spark、Apache Flink、Presto 和許多其他開放原始碼工具和架構。使用 Dataproc 以全球規模進行資料湖泊現代化、ETL / ELT 和安全數據資料學作業。Dataproc 也與多項 Google Cloud 服務完全整合，包括 [BigQuery](#)、[Cloud Storage](#)、[Vertex AI](#) 和 [Dataplex](#)。

Dataproc 提供三種版本：

- [無伺服器型 Dataproc](#) 可讓您執行 PySpark 工作，不必設定基礎架構和自動調度資源。無伺服器型 Dataproc 支援 PySpark 批次工作負載和工作階段 / 筆記本。
- [Google Compute Engine 上的 Dataproc](#) 可讓您管理 Hadoop YARN 叢集，處理以 YARN 為基礎的 Spark 工作負載，以及 Flink 和 Presto 等開放原始碼工具。您可以視需要垂直或水平擴充雲端叢集，包括自動調度資源。
- [Google Kubernetes Engine 上的 Dataproc](#) 可讓您在 GKE 基礎架構中設定 Dataproc 虛擬叢集，以提交 Spark、PySpark、SparkR 或 Spark SQL 工作。

在本程式碼研究室中，您將瞭解幾種不同的 Dataproc Serverless 使用方式。

Apache Spark 最初是為了在 Hadoop 叢集上執行而建構，並使用 YARN 做為資源管理工具。維護 Hadoop 叢集需要特定專業知識，並確保叢集上的許多不同旋鈕都已正確設定。此外，使用者還必須設定另一組 Spark 旋鈕。因此，開發人員往往花費更多時間設定基礎架構，而非處理 Spark 程式碼本身。

Dataproc Serverless 可免除手動設定 Hadoop 叢集或 Spark 的需求。Dataproc Serverless 不會在 Hadoop 上執行，而是使用自己的[動態資源分配](#)功能來判斷資源需求 (包括自動調度資源)。您仍可使用 Dataproc Serverless 自訂一小部分 [Spark 屬性](#)，但大多數情況下不需要調整這些屬性。

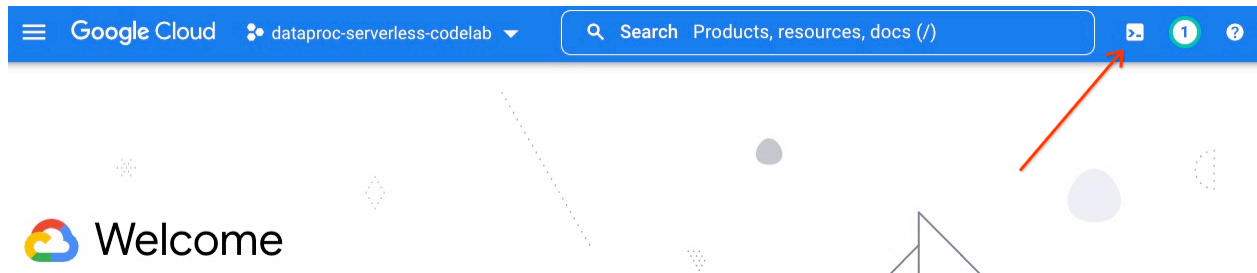
步驟 2 · 約 7 分鐘

2. 設定

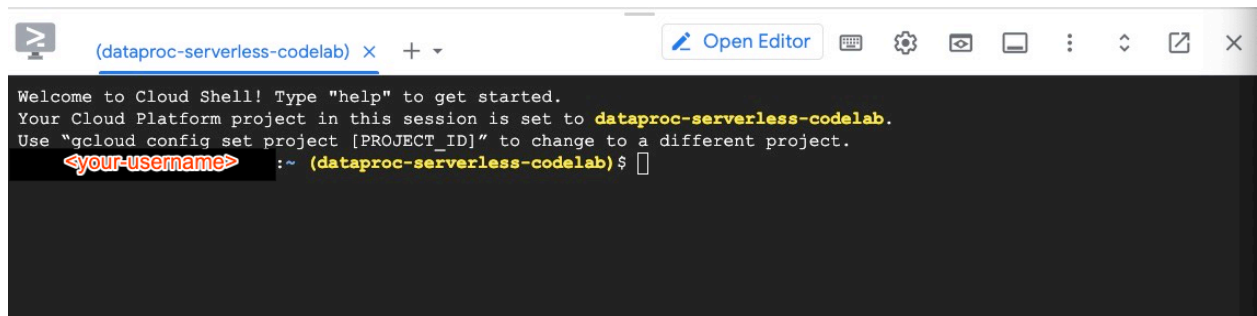
首先，請設定環境和本程式碼研究室使用的資源。

[建立](#) Google Cloud 專案。您可以使用現有專案。

按一下 [Cloud 控制台](#) 工具列中的按鈕，開啟 Cloud Shell。



Cloud Shell 提供立即可用的殼層環境，可用於本程式碼研究室。



Cloud Shell 預設會設定專案名稱。請再次執行 `echo $GOOGLE_CLOUD_PROJECT`，確認結果。如果輸出內容中未顯示專案 ID，請設定專案 ID。



```
export GOOGLE_CLOUD_PROJECT=<your-project-id>
```

為資源設定 Compute Engine [區域](#)，例如 `us-central1` 或 `eu-west-2`。



```
export REGION=<your-region>
```

啟用 API

本程式碼研究室會使用下列 API：

- BigQuery
- Dataproc

啟用必要的 API。這項作業大約需要一分鐘，完成後畫面會顯示成功訊息。

```
gcloud services enable bigquery.googleapis.com
gcloud services enable dataproc.googleapis.com
```

設定網路存取權

由於 Spark 驅動程式和執行程式只有私人 IP，因此 Dataproc Serverless 要求您在執行 Spark 工作的區域中啟用 [Google Private Access](#)。執行下列指令，在 `default` 子網路中啟用這項功能。

```
gcloud compute networks subnets update default \
  --region=${REGION} \
  --enable-private-ip-google-access
```

您可以透過下列指令驗證是否已啟用私人 Google 存取權，輸出內容為 `True` 或 `False`。

```
gcloud compute networks subnets describe default \
  --region=${REGION} \
  --format="get(privateIpGoogleAccess)"
```

建立儲存空間值區

建立儲存空間 bucket，用於儲存本程式碼研究室建立的資產。

為值區命名。值區名稱在[所有使用者中不得重複](#)。

```
export BUCKET=<your-bucket-name>
```

在您打算執行 Spark 工作的地區建立 bucket。

```
gsutil mb -l ${REGION} gs://${BUCKET}
```

您可以在 Cloud Storage [控制台](#)中看到 bucket。您也可以執行 `gsutil ls` 來查看 bucket。

建立永久記錄伺服器

[Spark UI](#) 提供豐富的偵錯工具，以及 Spark 工作的深入分析資料。如要查看已完成的 Dataproc Serverless 工作 Spark UI，您必須建立單一節點 Dataproc 叢集，做為[永久記錄伺服器](#)。

設定永久記錄伺服器的名稱。

```
PHS_CLUSTER_NAME=my-phs
```

執行下列指令。

```
gcloud dataproc clusters create ${PHS_CLUSTER_NAME} \
  --region=${REGION} \
  --single-node \
  --enable-component-gateway \
  --properties=spark:spark.history.fs.logDirectory=gs://${BUCKET}/phs/*/spark-job-history
```

本程式碼研究室稍後會更詳細地介紹 Spark UI 和永久記錄伺服器。

3. 使用 Dataproc Batches 執行無伺服器 Spark 工作

在本範例中，您將使用[紐約市 \(NYC\) Citi Bike 行程公開資料集](#)的一組資料。NYC Citi Bikes 是紐約市的付費單車共享系統。您將執行一些簡單的轉換，並列印出十大最熱門的 Citi Bike 車站 ID。此外，這個範例也特別使用開放原始碼的[spark-bigquery-connector](#)，在 Spark 和 BigQuery 之間順暢地讀取及寫入資料。

複製下列 Github 存放區，然後將 `cd` 複製到包含 `citibike.py` 檔案的目錄。

```
git clone https://github.com/GoogleCloudPlatform/devrel-demos.git
cd devrel-demos/data-analytics/next-2022-workshop/dataproc-serverless
```

`citibike.py`

```
import sys

from pyspark.sql import SparkSession
from pyspark.sql.functions import col
from pyspark.sql.types import BooleanType

if len(sys.argv) == 1:
    print("Please provide a GCS bucket name.")

bucket = sys.argv[1]
table = "bigquery-public-data:new_york_citibike.citibike_trips"

spark = SparkSession.builder \
    .appName("pyspark-example") \
    .config("spark.jars","gs://spark-lib/bigquery/spark-bigquery-with- \
dependencies_2.12-0.26.0.jar") \
    .getOrCreate()

df = spark.read.format("bigquery").load(table)

top_ten = df.filter(col("start_station_id") \
    .isNotNull()) \
    .groupBy("start_station_id") \
    .count() \
    .orderBy("count", ascending=False) \
    .limit(10) \
    .cache()

top_ten.show()

top_ten.write.option("header",
True).csv(f"gs://{bucket}/citibikes_top_ten_start_station_ids")
```

使用 [Cloud SDK](#) 將工作提交至無伺服器 Spark，Cloud Shell 預設提供這項工具。在殼層中執行下列指令，透過 Cloud SDK 和 [Dataproc Batches API](#) 提交無伺服器 Spark 工作。

```
gcloud dataproc batches submit pyspark citibike.py \
  --batch=citibike-job \
  --region=${REGION} \
  --deps-bucket=gs://${BUCKET} \
  --jars=gs://spark-lib/bigquery/spark-bigquery-with-dependencies_2.12-0.26.0.jar \
  --history-server-
cluster=projects/${GOOGLE_CLOUD_PROJECT}/regions/${REGION}/clusters/${PHS_CLUSTER_NAME} \
  -- ${BUCKET}
```

詳細說明如下：

- `gcloud dataproc batches submit` 參照 [Dataproc Batches API](#)。
- `pyspark` 表示您要提交 PySpark 工作。
- `--batch` 是工作的名稱。如未提供，系統會使用隨機產生的 UUID。
- `--region=${REGION}` 是處理作業的地理區域。
- `--deps-bucket=${BUCKET}` 是上傳本機 Python 檔案的位置，檔案上傳後即可在 Serverless 環境中執行。
- `--jars=gs://spark-lib/bigquery/spark-bigquery-with-dependencies_2.12-0.26.0.jar` 在 Spark 執行階段環境中包含 [spark-bigquery-connector](#) 的 JAR。
- `--history-server-cluster=projects/${GOOGLE_CLOUD_PROJECT}/regions/${REGION}/clusters/${PHS_CLUSTER}` 是永久記錄伺服器的完整名稱。系統會在這裡儲存 Spark 事件資料 (與控制台輸出內容不同)，並透過 Spark UI 顯示。
- 結尾的 `--` 表示之後的任何內容都是程式的執行階段引數。在本例中，您會依工作要求提交 bucket 名稱。

批次提交後，系統會輸出下列內容。

```
Batch [citibike-job] submitted.
```

幾分鐘後，您會看到以下輸出內容，以及作業的中繼資料。

```
+-----+-----+
|start_station_id| count|
+-----+-----+
|                |551078|
|                |423334|
|                |403795|
|                |384116|
|                |372255|
|                |367194|
|                |344546|
|                |330378|
|                |318700|
|                |311403|
+-----+-----+
```

```
Batch [citibike-job] finished.
```


在下一節中，您將瞭解如何找出這項工作的記錄。

其他功能

使用無伺服器 Spark 時，您有更多執行工作的方式。

- 您可以建立工作執行的[自訂 Docker 映像檔](#)。這是納入 Python 和 R 程式庫等其他依附元件的絕佳方式。
 - 您可以將 [Dataproc Metastore](#) 執行個體連線至工作，以存取 Hive 中繼資料。
 - Dataproc Serverless 支援部分 [Spark 屬性](#) 設定，可讓您進一步控管。
-

4. Dataproc 指標和觀測能力

[Dataproc Batches 控制台](#)會列出所有 Dataproc Serverless 工作。在控制台中，您會看到每個工作的**批次 ID**、**位置**、**狀態**、**建立時間**、**經過時間**和**類型**。按一下工作的「Batch ID」(批次 ID)，即可查看更多相關資訊。

這個頁面會顯示「Monitoring」(監控)等資訊，讓您瞭解工作在一段時間內使用了多少**批次 Spark 執行器**(表示自動調度資源的程度)。

在「詳細資料」分頁中，您會看到更多工作相關中繼資料，包括隨工作提交的任何引數和參數。

您也可以透過這個頁面存取所有記錄。執行 Dataproc Serverless 工作時，系統會產生三種不同的記錄：

- 服務層級
- 控制台輸出內容
- Spark 事件記錄

服務層級：包含 Dataproc Serverless 服務產生的記錄。包括 Dataproc Serverless 要求額外 CPU 來自動調度資源。按一下「查看記錄」即可查看這些記錄，系統會開啟 [Cloud Logging](#)。

控制台輸出內容會顯示在「Output」(輸出內容)下方。這是工作產生的輸出內容，包括 Spark 在工作啟動時列印的中繼資料，或納入工作中的任何 print 陳述式。

您可以透過 Spark UI 存取 **Spark 事件記錄**。由於您為 Spark 工作提供永久記錄伺服器，因此可以按一下「View Spark History Server」(查看 Spark 記錄伺服器)存取 Spark UI，其中包含先前執行的 Spark 工作資訊。如要進一步瞭解 Spark UI，請參閱官方 [Spark 說明文件](#)。

步驟 5 · 約 5 分鐘

5. Dataproc 範本：BigQuery -> Cloud Storage

Dataproc 範本是開放原始碼工具，可進一步簡化雲端資料處理工作。這些範本是 Dataproc 無伺服器的封裝函式，包含許多資料匯入和匯出工作的範本，包括：

- BigQuerytoGCS 和 GCStoBigQuery
- GCStoBigTable
- GCStoJDBC 和 JDBCtoGCS
- HivetoBigQuery
- MongotoGCS 和 GCStoMongo

完整清單請參閱 [README](#)。

在本節中，您將使用 Dataproc 範本，將資料從 [BigQuery](#) 匯出至 [GCS](#)。

建立存放區的本機複本

複製存放區的本機複本，並變更為 `python` 資料夾。

```
git clone https://github.com/GoogleCloudPlatform/dataproc-templates.git
cd dataproc-templates/python
```

設定環境

現在要設定環境變數。Dataproc 範本會使用專案 ID 的環境變數 `GCP_PROJECT`，因此請將此變數設為 `GOOGLE_CLOUD_PROJECT`。

```
export GCP_PROJECT=${GOOGLE_CLOUD_PROJECT}
```

您應在先前的環境中設定 [區域](#)。如果沒有，請在這裡設定。

```
export REGION=<region>
```

Dataproc 範本會使用 [spark-bigquery-connector](#) 處理 BigQuery 工作，並要求 URI 納入環境變數 `JARS`。設定 `JARS` 變數。

```
export JARS="gs://spark-lib/bigquery/spark-bigquery-with-dependencies_2.12-0.26.0.jar"
```

設定範本參數

設定服務要使用的暫存值區名稱。

```
export GCS_STAGING_LOCATION=gs://${BUCKET}
```

接著，您將設定一些工作專屬變數。輸入資料表時，您會再次參照 BigQuery NYC Citibike 資料集。

```
BIGQUERY_GCS_INPUT_TABLE=bigquery-public-data.new_york_citibike.citibike_trips
```

你可以選擇 `csv`、`parquet`、`avro` 或 `json`。在本程式碼研究室中，請選擇 `CSV`，下一節將說明如何使用 `Datapro` 範本轉換檔案類型。

```
BIGQUERY_GCS_OUTPUT_FORMAT=csv
```

將輸出模式設為 `overwrite`。你可以選擇 `overwrite`、`append`、`ignore` 或 `errorifexists`。

```
BIGQUERY_GCS_OUTPUT_MODE=overwrite
```

將 GCS 輸出位置設為 bucket 中的路徑。

```
BIGQUERY_GCS_OUTPUT_LOCATION=gs://${BUCKET}/BQtoGCS
```

執行範本

指定下方範本並提供您設定的輸入參數，即可執行 `BIGQUERYTOGCS` 範本。

```
./bin/start.sh \  
-- --template=BIGQUERYTOGCS \  
    --bigquery.gcs.input.table=${BIGQUERY_GCS_INPUT_TABLE} \  
    --bigquery.gcs.output.format=${BIGQUERY_GCS_OUTPUT_FORMAT} \  
    --bigquery.gcs.output.mode=${BIGQUERY_GCS_OUTPUT_MODE} \  
    --bigquery.gcs.output.location=${BIGQUERY_GCS_OUTPUT_LOCATION}
```

輸出內容會相當雜亂，但大約一分鐘後，您會看到下列內容。

```
Batch [5766411d6c78444cb5e80f305308d8f8] submitted.  
...  
Batch [5766411d6c78444cb5e80f305308d8f8] finished.
```

執行下列指令，即可確認檔案是否已生成。

```
gsutil ls ${BIGQUERY_GCS_OUTPUT_LOCATION}
```

根據預設，`Spark` 會視資料量寫入多個檔案。在本例中，系統會產生約 30 個檔案。`Spark` 輸出檔案名稱採用以下格式：開頭為 `part`，後面接著五個數字 (代表元件編號) 和一組雜湊字串。如果資料量很大，`Spark` 通常會寫入多個檔案。檔案名稱範例：`part-00000-cbf69737-867d-41cc-8a33-6521a725f7a0-c000.csv`。

步驟 6 · 約 5 分鐘

6. Dataproc 範本：CSV 轉 Parquet

您現在要使用 Dataproc 範本，透過 [GCStoGCS](#) 將 GCS 中的資料從一種檔案類型轉換為另一種。這個範本使用 SparkSQL，並提供選項，讓您也能提交 SparkSQL 查詢，在轉換期間進行處理，以利額外處理。

確認環境變數

確認已設定前一節中的 `GCP_PROJECT`、`REGION` 和 `GCS_STAGING_BUCKET`。

```
echo ${GCP_PROJECT}
echo ${REGION}
echo ${GCS_STAGING_LOCATION}
```

設定範本參數

現在要為「`GCStoGCS`」設定配置參數。首先，請指定輸入檔案的位置。請注意，這是目錄，不是特定檔案，因為系統會處理目錄中的所有檔案。請將此值設為 `BIGQUERY_GCS_OUTPUT_LOCATION`。

```
GCS_TO_GCS_INPUT_LOCATION=${BIGQUERY_GCS_OUTPUT_LOCATION}
```

設定輸入檔案的格式。

```
GCS_TO_GCS_INPUT_FORMAT=csv
```

設定所需的輸出格式。您可以選擇 Parquet、JSON、Avro 或 CSV。

```
GCS_TO_GCS_OUTPUT_FORMAT=parquet
```

將輸出模式設為 `overwrite`。你可以選擇 `overwrite`、`append`、`ignore` 或 `errorifexists`。

```
GCS_TO_GCS_OUTPUT_MODE=overwrite
```

設定輸出位置。

```
GCS_TO_GCS_OUTPUT_LOCATION=gs://${BUCKET}/GCStoGCS
```

執行範本

執行 `GCStoGCS` 範本。

```
./bin/start.sh \  
-- --template=GCSTOGCS \  
    --gcs.to.gcs.input.location=${GCS_TO_GCS_INPUT_LOCATION} \  
    --gcs.to.gcs.input.format=${GCS_TO_GCS_INPUT_FORMAT} \  
    --gcs.to.gcs.output.format=${GCS_TO_GCS_OUTPUT_FORMAT} \  
    --gcs.to.gcs.output.mode=${GCS_TO_GCS_OUTPUT_MODE} \  
    --gcs.to.gcs.output.location=${GCS_TO_GCS_OUTPUT_LOCATION}
```

輸出內容會相當雜亂，但大約一分鐘後，您應該會看到類似下方的成功訊息。

```
Batch [c198787ba8e94abc87e2a0778c05ec8a] submitted.  
...  
Batch [c198787ba8e94abc87e2a0778c05ec8a] finished.
```

執行下列指令，即可確認檔案是否已生成。

```
gsutil ls ${GCS_TO_GCS_OUTPUT_LOCATION}
```

您也可以使用這個範本，將 `gcs.to.gcs.temp.view.name` 和 `gcs.to.gcs.sql.query` 傳遞至範本，提供 SparkSQL 查詢，以便在將資料寫入 GCS 前，先對資料執行 SparkSQL 查詢。

7. 清除所用資源

完成本程式碼研究室後，為避免系統向您的 GCP 帳戶收取不必要的費用，請按照下列步驟操作：

1. 刪除您建立的環境的 [Cloud Storage bucket](#)。

```
gsutil rm -r gs://${BUCKET}
```

2. 刪除用於永久記錄伺服器的 [Dataproc 叢集](#)。

```
gcloud dataproc clusters delete ${PHS_CLUSTER_NAME} \
  --region=${REGION}
```

3. 刪除 Dataproc Serverless 工作。前往 [Batches 控制台](#)，點按要刪除的每個作業旁邊的方塊，然後點按「DELETE」（刪除）。

如果您專為本程式碼研究室建立了專案，也可以選擇刪除該專案：

1. 前往 GCP 主控台的「Projects」（專案）頁面。
2. 在專案清單中，選取要刪除的專案，然後按一下「刪除」。
3. 在方塊中輸入專案 ID，然後按一下「關閉」刪除專案。

警告：刪除專案會出現以下結果：

- **專案中的所有內容都會遭到刪除。**如果您之前使用現有的專案來進行本教學課程，當您刪除該專案時，也會一併刪除您在該專案中完成的所有其他工作。
- **自訂專案 ID 不復存在。**您建立這個專案時，可能已建立日後使用的自訂專案 ID。如要保留使用該專案 ID 的網址 (例如 [appspot.com](#) 網址)，請刪除在該專案中選取的資源，而不是刪除整個專案。

步驟 8 · 約 2 分鐘

8. 後續步驟

下列資源提供其他方式，讓您充分運用 Serverless Spark：

- 瞭解如何使用 [Cloud Composer 自動化調度管理](#) Dataproc Serverless 工作流程。
 - 瞭解如何將 Dataproc Serverless 與 [Kubeflow 管道](#)整合。
-